Email Automation Campaign

n8n Workflow Architecture & Implementation Guide

Overview: This document outlines the transition of the CEO Email Automation Campaign from local Python scripts to a node-based architecture using **n8n**. By migrating to n8n, the system gains native IMAP polling, visual debugging, reliable rate-limiting without thread locks, and seamless API integrations.

Core Infrastructure Requirement: It is highly recommended to migrate the local `ceo_data.xlsx` file to **Google Sheets**. n8n natively integrates with Google Sheets, allowing for instantaneous row lookups and eliminating local file permission issues.

Module 1: Data Scraper & Email Verifier

This workflow pulls raw CEO data, generates potential email patterns, verifies them via API, and updates the master list.

Node Pipeline

1. Schedule Trigger

Set to trigger manually or on a weekly CRON schedule.

2. Google Sheets (Read)

Fetches the "CEO Master List".

3. Code Node (JavaScript)

Replaces the Python `clean()` and `generate_candidates()` logic. Extracts names and generates email variations.

4. HTTP Request Node

Connects to Snov.io / Apollo / Hunter API to verify the generated candidates.

5. Google Sheets (Update)

Writes the verified email back to the sheet and sets `Email Valid = TRUE`.

Code Node Payload (JavaScript)

```
// Clean names and generate email patterns for verification
const patterns = [
  "{first}.{last}", "{first}", "{f}{last}", "{first}{last}",
  "{first}_{last}", "{first}-{last}", "{f}.{last}", "{last}.{first}"
];

for (const item of $input.all()) {
  let fullName = item.json['Full Name'] || "";
  let domain = item.json['Company Domain'] || "";

  // Extract First, Last, and Initials
  let parts = fullName.trim().split(" ");
  let first = parts[0] ? parts[0].toLowerCase() : "";
  let last = parts.length > 1 ? parts[parts.length - 1].toLowerCase() : "";
  let f = first ? first[0] : "";
  let l = last ? last[0] : "";

  // Generate pattern candidates
  let candidates = [];
  if (domain && first) {
    for (let p of patterns) {
      let local = p.replace("{first}", first)
                   .replace("{last}", last)
                   .replace("{f}", f)
                   .replace("{l}", l);
      candidates.push(`${local}@${domain}`);
    }
  }

  // Add clean data back to the JSON payload for the next node
  item.json['first_name'] = first.charAt(0).toUpperCase() + first.slice(1);
  item.json['last_name'] = last;
  item.json['email_candidates'] = candidates;
}

return $input.all();
```

Module 2: Rate-Limited Bulk Mailer

Replaces the `bulk_mailer.py` script. n8n handles the 72-second delay natively using a Wait node, preventing terminal lockups and ensuring CAN-SPAM compliance.

Node Pipeline

1. Manual Trigger

Initiates the bulk send.

2. Google Sheets (Read & Filter)

Fetches targets where **Email Valid = TRUE**.

3. Loop Node

Configured to process exactly **1 item at a time**.

4. Send Email Node (SMTP)

Connects to SendGrid. Injects variables natively (e.g., `{{ $json.first_name }}`) into the Subject and HTML Body.

5. Wait Node

Configured to **Resume After 72 Seconds**. The output connects back to the Loop Node to pull the next target.

Module 3: The Auto-Reply "Gatekeeper"

Replaces `auto_reply.py`. This event-driven workflow only executes when an email arrives, eliminating the need for a continuous `while True` polling loop.

Node Pipeline

1. Email Read (IMAP) Trigger

Connects to Gmail. Monitors **INBOX** for **Unread Messages Only**. Resolves content as HTML/Text.

2. Google Sheets (Lookup Row)

Searches the CEO Sheet. Column to Search: `Email Address`. Value to Search: `{{ $json.from.address }}`.

3. IF Node (The Gatekeeper)

Condition: Does `{{ $json["Full Name"] }}` exist?
False Path: Do nothing (safely ignores non-CEO emails like YouTube/Spam).
True Path: Proceed to the next step.

4. Send Email Node (Auto-Reply)

Sends the Calendly link.
To: `{{ $node["Email Read (IMAP)"].json.from.address }}`
Subject: `Re: {{ $node["Email Read (IMAP)"].json.subject }}`

5. Google Sheets (Append Row)

Logs the event to the "Replies Log" sheet with a timestamp (`{{ $now }}`).